תרגיל 1: תנאים, לולאות, פונקציות ומבני נתונים בסיסיים

תאריך פרסום:	27.10.2026
תאריך הגשה:	08.11.2026, עד שעה 23:59
מתרגל\ת אחראי\ת:	טל מלול
משקל תרגיל:	2.5 נקודות
מטרות העבודה:	שימוש בתנאים, לולאות, פונקציות, רשימות, טפלים ומילונים.

דגשים:

1. במימוש תרגיל זה עליכם להשתמש אך ורק בפקודות הבאות, כלומר אין להשתמש בפקודות אחרות, אפילו אם הן נלמדו בשיעור.
   a. תנאים: if, elif, else, in וכל פעולות ההשוואה שנלמדו בכיתה.
   b. לולאות: for, while, continue, break, in, range, וכו'.
   c. פונקציות: def, return ערכים דיפולטיים, וכו'.
   d. ניתן להשתמש במשתנים מספריים, בוליאניים, תווים, מחרוזות, רשימות, טפלים ומלונים. אין להשתמש בסוגי משתנים אחרים.
   e. הפונקציות האריתמטיות (+, -, ** וכו') והלוגיות (and, or וכו') של פייתון.
   f. מתוך הפונקציות המובנות של פייתון ניתן להשתמש רק באלו: len, chr, str, ord, int, bool, float, list, dict, pow, type, range, min, max, print, sum, enumerate, round.
   g. כל השיטות של String (אלא אם כן נאמר אחרת).
   h. מותר להשתמש בשיטות pop, insert, index ו-append של list. ובשיטות get, keys, values, items ו-update של dict.
2. ניתן ואף מומלץ להוסיף פונקציות עזר.
3. חובה לכתוב docstring (כפי שלמדתם בתרגול) לכל פונקציה.

רקע

חברת רכבות המפעילה מספר רכבות ברחבי הארץ מעוניינת לפתח מערכת ממוחשבת לניהול לוחות הזמנים של הרכבות. לכל רכבת מוגדר לוח זמנים המתאר את זמני ההגעה והיציאה מכל תחנה ומהווה את מסלול הנסיעה שלה. **כל רכבת עוברת בכלל התחנות אולם לא בהכרח עוצרת בהן**. בעבודה זו תממשו תוכנית לניהול פעילותה של חברת הרכבות.

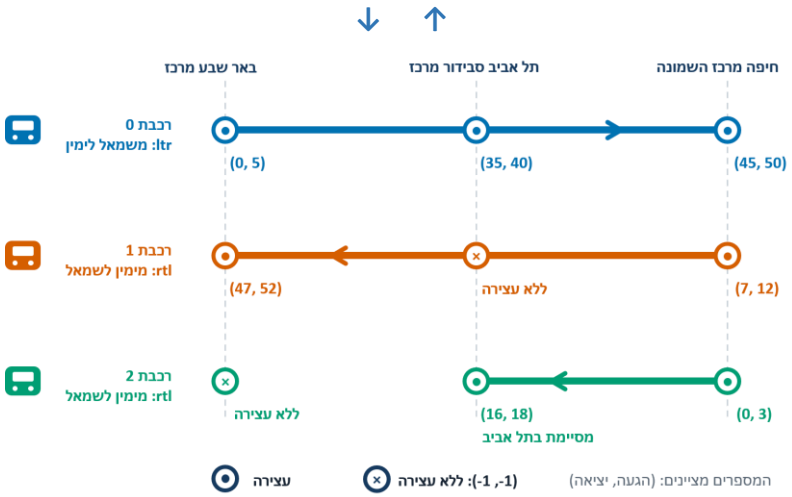
להלן הסבר על צורת ייצוג הרכבות ולוחות הזמנים אשר ישמש אותכם בפיתוח המערכת:

- **לוח הזמנים של רכבת** מיוצג באמצעות רשימה של tuples, שאורכה שווה למספר התחנות. כל tuple ברשימה מכיל שני ערכים מטיפוס int, המייצגים את זמן הגעת הרכבת לתחנה ואת זמן יציאתה ממנה, לפי הסדר: (arrival, departure). המספרים אינם מייצגים שעות בשעון, אלא יחידות זמן מופשטות, כאשר מספר קטן יותר מייצג זמן מוקדם יותר. לדוגמה, אם הרכבת מגיעה לתחנה הראשונה ביחידת הזמן 9 ויוצאת ממנה ביחידת הזמן 10, ה־tuple המייצג את התחנה בלוח הזמנים יהיה (10, 9).
- אם רכבת אינה עוצרת בתחנה מסוימת, ערך ה-tuple המתאים יהיה $(-1, -1)$.
- **לוחות הזמנים של כל הרכבות בחברה** מאוחסן כרשימה מקוננת בגודל $n \times m$ (מטריצה) כאשר n יהיה כמספר הרכבות ו-m יהיה כמספר התחנות בהן עוצרות או עוברות הרכבות. למשל בהינתן שלחברה יש 3 רכבות (ראו את הטבלה והאיור למטה), לוח הזמנים של כל הרכבות יוצג באופן הבא:

```
[[(0, 5), (35, 40), (45, 50)], [(47, 52), (-1, -1), (7, 12)], [(-1, -1), (16, 18), (0, 3)]]
```

- לכל רכבת מוגדר כיוון תנועה, המיוצג באמצעות מחרוזת. כיוון התנועה קובע את הסדר שבו יש לקרוא את סדר התחנות בלוח הזמנים של הרכבת:
  - 'ltr' (left-to-right, משמאל לימין) – הרכבת נוסעת מתחנות בעלות אינדקס נמוך לתחנות בעלות אינדקס גבוה. כלומר, יש לקרוא את לוח הזמנים מתחילתו לסופו.
  - 'rtl' (right-to-left, מימין לשמאל) – הרכבת נוסעת מתחנות בעלות אינדקס גבוה לתחנות בעלות אינדקס נמוך. כלומר, יש לקרוא את לוח הזמנים מסופו לתחילתו.

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת 0 :ltr	(0, 5)	(35, 40)	(45, 50)
רכבת 1 :rtl	(47, 52)	(-1, -1)	(7, 12)
רכבת 2 :rtl	(-1, -1)	(16, 18)	(0, 3)



ניתן להניח כי הקלט תקין וכל הרשימות הפנימיות הן באותו האורך m.

העבודה מחולקת לשלושה חלקים: **בחלק הראשון** תממשו פונקציות לחישוב הנחות במחירי כרטיסים ולבדיקת קיומן של נסיעות לפי הצורך, **בחלק השני** תממשו פונקציות לניהול לוחות הזמנים של הרכבות, לרבות זיהוי התנגשויות ועדכון זמני הנסיעה, **ובחלק השלישי** תאגדו את נתוני החברה במבנה נתונים של מילון ותממשו פונקציה המתכננת נסיעה ישירה עבור קבוצת נוסעים. שימו לב שתלמדו על מילונים ב-1.11, אך ניתן להתחיל לממש את החלקים הראשון והשני לפני כן.

חלק 1: מחירים ובדיקת נסיעות

סעיף א': חישוב מחיר כרטיס

חברת הרכבות מציעה לנוסעיה מגוון והנחות במחירי הכרטיסים. ההנחה מיוצגת באמצעות אחוז המחושב מתוך מחיר הכרטיס הבסיסי. מחיר הכרטיס לאחר ההנחה ייקבע בהתאם לאחוז ההנחה שהנוסע זכאי לו. כמו כן, אם נוסע זכאי ליותר מהנחה אחת, תחול עליו ההנחה הגבוהה ביותר בלבד - כלומר ההנחות אינן מצטברות. להלן כללי ההנחות אותן מציעה החברה:

- נוסע שגילו קטן מ-5 נוסע ללא תשלום.
- נוסע שגילו בין 5 ל-17 (כולל 17), או נוסע שגילו 65 ומעלה (אזרח ותיק), מקבל הנחה של 50%.
- נוסע שמחזיק תעודת סטודנט וגילו 18 ומעלה מקבל הנחה של 20%.
- בכל מקרה אחר, על הנוסע לשלם את מחיר הכרטיס המלא.

ממשו את הפונקציה:

$$calculate_ticket_price(base_price, passenger_age, is_student)$$

הפונקציה מקבלת בקלט 3 ארגומנטים:

- **base_price** – משתנה מטיפוס float מספר חיובי המציין את מחיר הכרטיס המלא.
- **passenger_age** –משתנה מטיפוס int מספר טבעי (גדול מ-0) המציין את גיל נוסע.
- **is_student –** משתנה בוליאני המציין האם הנוסע סטודנט**.**

על הפונקציה לחשב את ההטבה שמגיעה לנוסע ולהחזיר את מחיר הכרטיס **לאחר ההנחה** שאליה זכאי הנוסע:
מספר מטיפוס float, מעוגל לספרה אחת אחרי הנקודה העשרונית.
דוגמאות קלט פלט:

base_price	passenger_age	is_student	פלט
50	4	False	0.0
50	14	False	25.0
50	23	True	40.0
50	40	False	50.0
50	70	True	25.0

סעיף ב': בדיקה האם רכבת מאפשרת נסיעה ישירה

חברת הרכבות מציעה לנוסעיה לבדוק את האופציה לנסיעה ישירה ברכבת, ללא החלפות. **נסיעה ישירה** מוגדרת כנסיעה מתחנת המוצא לתחנת היעד באמצעות רכבת אחת, העוצרת בשתי התחנות ונוסעת ביניהן בכיוון המתאים.

בסעיף זה עליכם לממש פונקציה הבודקת עבור רכבת מסוימת, האם ניתן לבצע נסיעה ישירה מתחנת מוצא לתחנת יעד ע"ס מסלול הרכבת וכיוון הנסיעה:

$$is_direct_route(train_schedule, direction, start_station_index, destination_station_index)$$

הפונקציה מקבלת כקלט:

- **train_schedule** –לוח הזמנים של הרכבת.
- **direction** - מחרוזת המייצגת את כיוון הרכבת $'ltr'$ או $'rtl'$.
- **start_station_index** - מספר שלם בטווח $[0, m-1]$ המציין את אינדקס תחנת המוצא בתוך מסלול הנסיעה.
- **destination_station_index** - מספר שלם בטווח $[0, m-1]$ המציין את אינדקס תחנת היעד בתוך מסלול הנסיעה.

הפונקציה מחזירה True כאשר קיימת האפשרות לנסיעה ישירה בהתאם לתנאים הבאים:

- האינדקסים של תחנת המוצא ושל תחנת היעד נמצאים בטווח התקין של לוח הזמנים של הרכבת **train_schedule**.
- תחנת המוצא ותחנת היעד הן תחנות שונות.
- הרכבת מבצעת עצירה בשתי התחנות.
- סדר התחנות מתאים לכיוון נסיעת הרכבת:
  - באשר כיוון הנסיעה הוא משמאל לימין, אינדקס תחנת המוצא קטן מאינדקס תחנת היעד.
  - באשר כיוון הנסיעה הוא מימין לשמאל, אינדקס תחנת המוצא גדול מאינדקס תחנת היעד.
- זמן ההגעה לתחנת היעד אינו מוקדם מזמן היציאה מתחנת המוצא.

דוגמאות קלט פלט:

train_schedule	direction	start_station_index	destination_station_index	פלט
[(0,5),(20,25),(35,40)]	ltr	0	2	True
[(0,5),(-1,-1),(35,40)]	ltr	0	1	False

סעיף ג': חישוב משך זמן נסיעה

בסעיף זה עליכם לממש פונקציה המחשבת את משך הנסיעה של רכבת בין תחנת מוצא לתחנת יעד. ממשו את הפונקציה:

$$calculate_travel_time(train_schedule, direction, start_station_index, destination_station_index)$$

הפונקציה מקבלת כקלט:

- **train_schedule** – לוח הזמנים של הרכבת.
- **direction** - מחרוזת המייצגת את כיוון הרכבת ltr או rtl.
- **start_station_index** - מספר שלם בטווח $[0, m-1]$ המציין את אינדקס תחנת המוצא בתוך מסלול הנסיעה.
- **destination_station_index** - מספר שלם בטווח $[0, m-1]$ המציין את אינדקס תחנת היעד בתוך מסלול הנסיעה.

הפונקציה מחזירה את **משך זמן הנסיעה** (זמן ההגעה ליעד פחות זמן היציאה מן המוצא). אם הנסיעה אינה אפשרית, יש להחזיר -1.

דוגמאות קלט פלט:

train_schedule	direction	start_station_index	destination_station_index	פלט
[(0,5),(20,25),(35,40)]	ltr	0	2	30
[(0,5),(-1,-1),(35,40)]	ltr	0	1	-1

חלק 2: פונקציות בסיסיות לניהול לוחות זמנים

סעיף א': הוספת תחנה

לקראת תחילת שנת הלימודים, החליטה החברה להרחיב את פעילותיה ולהוסיף תחנות נוספות. בהתאם לייצוג לוחות הזמנים שתואר ברקע, כל תחנה מיוצגת באמצעות אותו אינדקס בכל לוחות הזמנים של הרכבות. לכן, הוספת תחנה חדשה מחייבת הוספת תא באותו אינדקס בכל לוחות הזמנים. בשלב זה אף רכבת עדיין אינה עוצרת בתחנה החדשה. לפיכך, התא החדש בכל לוח זמנים יאותחל לערך $(-1,-1)$. ממשו את הפונקציה:

$$initialize_station(schedules, station_index)$$

הפונקציה מקבלת בקלט:

- **schedules** – **לוחות הזמנים של כל הרכבות בחברה** (כמתואר מעלה).
- **station_index** – מספר שלם בטווח $[0, m]$ המציין את אינדקס התחנה החדשה שהולכת להתווסף בתוך המסלול של כל הרכבות.

על הפונקציה להוסיף תחנה חדשה בכל לוחות הזמנים של כל אחת מהרכבות בחברה, במיקום (**station_index**) ולאתחל את זמני ההגעה והיציאה של כל רכבת בתחנה החדשה לערך $(-1,-1)$. אם המיקום שניתן לפונקציה בקלט **station_index** אינו חוקי עבור רכבת מסוימת, הפונקציה לא תשנה את לוח הזמנים.

שימו לב: הפונקציה משנה את לוח הזמנים הקיים **ואינה מחזירה ערך חדש (מבצעת את השינוי in-place).**
לדוגמה:

1. קטע קוד:

```
>>> schedules = [[(0, 5), (10, 15), (20, 25)], [(2, 7), (12, 17), (22, 27)]]
>>> station_index = 1
>>> initialize_station(schedules, station_index)
>>> print(schedules)
[[(0, 5), (-1, -1), (10, 15), (20, 25)], [(2, 7), (-1, -1), (12, 17), (22, 27)]]
```

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת 0	(0, 5)	(10, 15)	(20, 25)
רכבת 1	(2, 7)	(12, 17)	(22, 27)

↓ initialize_station ↓

רכבת	באר שבע מרכז	ירושלים	תל אביב סבידור	חיפה מרכז השמונה
רכבת 0	(0, 5)	**(-1, -1)**	(10, 15)	(20, 25)
רכבת 1	(2, 7)	**(-1, -1)**	(12, 17)	(22, 27)

2. קטע קוד:

```
>>> schedules = [[(0, 5), (10, 15), (20, 25)], [(2, 7), (12, 17), (22, 27)]]
>>> station_index = -3
>>> initialize_station(schedules, station_index)
>>> print(schedules)
[[(0, 5), (10, 15), (20, 25)], [(2, 7), (12, 17), (22, 27)]]
```

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת 0	(0, 5)	(10, 15)	(20, 25)
רכבת 1	(2, 7)	(12, 17)	(22, 27)

↓ initialize_station ↓

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת 0	(0, 5)	(10, 15)	(20, 25)
רכבת 1	(2, 7)	(12, 17)	(22, 27)

סעיף ב': פיצול רכבת

חברת הרכבות החליטה לייעל את מערך הנסיעות באמצעות פיצול של מסלולי נסיעה קיימים. במסגרת השינוי, מסלול הנסיעה של רכבת מסוימת, כפי שהוא מיוצג בלוח הזמנים שלה, יחולק בנקודת פיצול לשני מסלולים עצמאיים, שכל אחד מהם יתבצע באמצעות רכבת אחרת. המסלול הראשון יכלול את התחנות הנמצאות לפני נקודת הפיצול, ואילו המסלול השני יתחיל בתחנה שבנקודת הפיצול ויכלול את התחנות שאחריה.

ממשו את הפונקציה:

$$split_schedule(schedules, train_directions, train_index, split_station)$$

הפונקציה מקבלת כקלט:

- **schedules** – לוח הזמנים של כל הרכבות בחברה.
- **train_directions** - רשימה של מחרוזות, מתאימה באורכה למספר הרכבות ב-schedules, שבה כל איבר מכיל את הערך $'ltr'$ או $'rtl'$ לפי כיוון הרכבת המתאימה.
- **train_index** – מספר שלם בטווח $[0, n-1]$ המציין את האינדקס של הרכבת ב-schedules שיש לפצל.
- **split_station** – מספר שלם בטווח $[1, m-1]$ המציין את האינדקס של התחנה בה מתבצע הפיצול.

הפונקציה מבצעת את פיצול מסלול הנסיעה של הרכבת שבאינדקס **train_index**. כל אחד מהמסלולים החדשים יתבצע באמצעות רכבת נפרדת וייוצג באמצעות לוח זמנים חדש. שני לוחות הזמנים החדשים יהיו באותו אורך כמו לוח הזמנים המקורי:

- מסלול הרכבת הראשונה יכיל את התחנות מהתא 0 עד **split_station** (לא כולל) עבור התחנות מאינדקס **split_station** ואילך יופיע הערך $(-1, -1)$.
- מסלול הרכבת השנייה יכיל את התחנות מ-**split_station** ואילך. עבור התחנות מאינדקס 0 ועד **split_station** (לא כולל) יופיע הערך $(-1, -1)$.
- שתי הרכבות החדשות יחליפו את הרכבת המקורית בלוח הזמנים של הרכבות בחברה, במיקום (**train_index** ו- **train_index+1** בהתאמה- תראו דוגמה למטה).
- כיוון הרכבת המקורית יישמר עבור שני המסלולים החדשים לכל רכבת. בהתאם לכך, הרשימה **train_directions** תעודכן באותם האינדקסים.

אם הערך של **split_station** או **train_index** מחוץ לטווח החוקי של לוח הזמנים של חברת הרכבות, הפונקציה לא תשנה דבר.

שימו לב: הפונקציה משנה את לוח הזמנים של הרכבות בחברה **במקום ואינה מחזירה ערך חדש (in-place).**

לדוגמה:

1. קטע קוד:

```
>>> schedules = [[(0, 5), (10, 15), (20, 25)], [(2, 7), (12, 17), (22, 27)]]
>>> train_directions = ['ltr', 'rtl']
>>> train_index = 0
>>> split_station = 2
>>> split_schedule(schedules,train_directions,train_index,split_station)
>>> print(schedules)
 [[(0, 5), (10, 15), (-1, -1)], [(-1, -1), (-1, -1), (20, 25)], [(2, 7), (12, 17), (22, 27)]]
>>> print(train_directions)
['ltr', 'ltr', 'rtl']
```

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת 0 :ltr	(0, 5)	(10, 15)	(20, 25)
רכבת 1 :rtl	(2, 7)	(12, 17)	(22, 27)

↓ split_schedule ↓

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת 0 :ltr	(0, 5)	(10, 15)	**(-1, -1)**
רכבת 1 :ltr	**(-1, -1)**	**(-1, -1)**	(20, 25)
רכבת 2 :rtl	(2, 7)	(12, 17)	(22, 27)

סעיף ג': זיהוי התנגשויות

אחד האתגרים בניהול מערך הרכבות הוא מניעת התנגשויות הנובעות מחפיפה בלוחות הזמנים של מספר רכבות. נניח כי בכל תחנה יכולה לשהות רכבת אחת בלבד בכל רגע נתון ונגדיר התנגשות כמצב שבו שתי רכבות שוהות באותה תחנה באותו זמן. שימו לב כי גם מצב שבו רכבת אחת עוזבת את התחנה בדיוק בזמן שרכבת אחרת מגיעה אליה נחשב להתנגשות.

בסעיף זה עליכם לממש פונקציה הבודקת האם קיימת התנגשות בין שתי רכבות בתחנה מסוימת, בהתאם לזמני ההגעה והיציאה שלהן. שימו לב כי בסעיף זה אין חשיבות לכיווני הנסיעה של הרכבות, וערך של $(-1, -1)$ אינו נחשב לשהייה בתחנה ולכן אינו יוצר התנגשות.

<u>לדוגמה:</u>
אם רכבת אחת שוהה בתחנה בין 5 ל־10, ורכבת אחרת שוהה באותה תחנה בין 9 ל־12 — קיימת התנגשות.
אם רכבת אחת יוצאת מהתחנה בזמן 10, ורכבת אחרת נכנסת לתחנה בדיוק באותו זמן— קיימת התנגשות.

ממשו את הפונקציה:

$$find_conflicts(schedules)$$

הפונקציה מקבלת כקלט:

- **schedules** – לוח הזמנים של הרכבות בחברה.

על הפונקציה לאתר התנגשויות בין רכבות לפי לוחות הזמנים שלהן, ותחזיר רשימה של כל ההתנגשויות בלוח הזמנים של הרכבות בחברה, התנגשות מיוצגת ע"י tuple שמכיל שלושה ערכים (**train_index_a, train_index_b, station**). כך ש-

- **train_index_a** – מספר שלם המייצג את האינדקס הנמוך מבין שתי הרכבות שביניהן זוהתה התנגשות.
- **train_index_b** – מספר שלם המייצג את האינדקס הגבוה מבין שתי הרכבות שביניהן זוהתה התנגשות.
- **station** – מספר שלם המייצג את אינדקס התחנה שבה זוהתה ההתנגשות בין הרכבות **train_index_a** ו-**train_index_b**.

אין להחזיר כפילויות: כל זוג רכבות יופיע לכל היותר פעם אחת לכל תחנה. סדר ההתנגשויות ברשימה המוחזרת אינו חשוב.

לדוגמה:

1. קטע קוד:

```
>>> schedules = [[(0, 5), (10, 15), (20, 25)], [(2, 7), (12, 17), (22, 27)], [(0, 3), (16, 18), (30, 35)]]
>>> print(find_conflicts(schedules))
[(0, 1, 0), (0, 1, 1), (0, 1, 2), (0, 2, 0), (1, 2, 0), (1, 2, 1)]
```

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת 0	(0, 5)	(10, 15)	(20, 25)
רכבת 1	(2, 7)	(12, 17)	(22, 27)
רכבת 2	(0, 3)	(16, 18)	(30, 35)

↓ **find_conflicts** ↓

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת 0	**(0, 5)**	**(10, 15)**	**(20, 25)**
רכבת 1	**(2, 7)**	**(12, 17)**	**(22, 27)**
רכבת 2	**(0, 3)**	**(16, 18)**	(30, 35)

סעיף ד': עיכוב רכבת

כחלק מהיערכות למניעת התנגשויות, חברת הרכבות מעוניינת לממש מנגנון שיאפשר עיכוב של זמני ההגעה והיציאה של רכבות. בסעיף זה, עליכם לממש פונקציה המעדכנת את זמני ההגעה והיציאה של רכבת מסוימת, החל מתחנה נתונה ועד לסוף מסלול נסיעתה, בהתאם לכיוון הנסיעה ולמשך העיכוב שנקבע. ממשו את הפונקציה:

$$delay_train(train_schedule, start_station, delay, direction = 'ltr')$$

הפונקציה מקבלת כקלט:

- **train_schedule** - לוח הזמנים של רכבת אחת.
- **start_station** – מספר שלם בטווח $[0, m-1]$ המציין את אינדקס התחנה ממנה מתחיל העיכוב, וניתן להניח שהוא בטווח.
- **delay** – מספר שלם חיובי, המציין בכמה זמן יש לעכב.
- **direction** - מחרוזת המייצגת את כיוון התפשטות העיכוב של הרכבת $'ltr'$ או $'rtl'$, כאשר ברירת המחדל היא $'ltr'$.

הפונקציה תעדכן את זמני ההגעה והיציאה (arrival, departure) של הרכבת בהתאם למשך העיכוב (**delay**), החל מהתחנה **start_station** ועד לסוף מסלול הנסיעה, בהתאם לכיוון הנסיעה (**direction**).
תחנה שבה הרכבת אינה עוצרת, המיוצגת באמצעות $(-1,-1)$, תישאר ללא שינוי גם אם היא נמצאת בתחום התחנות שעליהן חל העיכוב.
שימו לב: הפונקציה משנה את לוח הזמנים הקיים **במקום ואינה מחזירה ערך חדש (in-place).**
לדוגמה:

1. קטע קוד:

```
>>> train_schedule = [(2, 7), (12, 17), (22, 27)]
>>> start_station = 1
>>> delay = 5
>>> direction='ltr'
>>> delay_train(train_schedule,start_station,delay,direction)
>>> print(train_schedule)
[(2, 7), (17, 22), (27, 32)]
```

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת	(2, 7)	(12, 17)	(22, 27)

↓ delay_train ↓

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת	(2, 7)	**(17, 22)**	**(27, 32)**

2. קטע קוד:

```
>>> train_schedule = [(2, 7), (17, 22), (-1, -1)]
>>> start_station = 2
>>> delay = 3
>>> direction='rtl'
>>> delay_train(train_schedule,start_station,delay,direction)
>>> print(train_schedule)
[(5, 10), (20, 25), (-1, -1)]
```

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת	(2, 7)	(17, 22)	(-1, -1)

↓ delay_train ↓

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
רכבת	**(5, 10)**	**(20, 25)**	(-1, -1)

סעיף ה': פתרון התנגשויות

חברת הרכבות מעוניינת למנוע התנגשויות באמצעות הפונקציות שמימשתם בסעיפים הקודמים. לשם כך, עליכם לממש פונקציה המזהה התנגשויות ומטפלת בהן באמצעות עדכון לוחות הזמנים שלהן, עד שלא יוותרו התנגשויות. ממשו את הפונקציה:

$$resolve_conflicts(schedules, train_directions, delay)$$

הפונקציה מקבלת כקלט:

- **schedules** - לוח הזמנים של הרכבות בחברה.
- **train_directions** - רשימה של מחרוזות, שבה כל איבר מכיל את הערך $'ltr'$ או $'rtl'$ לפי כיוון רכבת.
- **delay** – מספר שלם חיובי, המציין את זמן העיכוב.

הפונקציה תטפל בהתנגשויות בין רכבות באופן איטרטיבי, בהתאם לכללים המפורטים בהמשך, תוך עדכון לוחות הזמנים של הרכבות במקום (in-place), עד שלא נותרות התנגשויות. על הפונקציה להחזיר מספר שלם אי-שלילי המייצג את מספר ההתנגשויות שנמנעו.

עליכם לממש את הפונקציה בהתאם לכללים הבאים:

- יש לטפל בהתנגשויות באופן איטרטיבי, באמצעות הפונקציות **`find_conflicts`** ו-**`delay_train`** שמימשתם בסעיפים הקודמים.
- בכל שלב יש לבחור את ההתנגשות לטיפול בהתאם לסדר האינדקסים של הרכבות והתחנות. סדר העדיפויות לטיפול נקבע לפי השלשה (**train_index_a**, **train_index_b**, **station**), בסדר לקסיקוגרפי עולה. כלומר, תחילה נבחרת ההתנגשות עם **train_index_a** הנמוך ביותר. במידה ויש מספר התנגשויות עם אותו מספר של **train_index_a**, ניתנת העדיפות להתנגשות עם **train_index_b** הנמוך ביותר. במידה ועדיין קיימות מספר התנגשויות שחולקות את **train_index_a** ו-**train_index_b** , נבחרת ההתנגשות עם ערך ה-station הנמוך ביותר.
- לאחר בחירת ההתנגשות לטיפול, יש לעכב את הרכבת בעלת האינדקס הנמוך יותר (כלומר את הרכבת בעלת **train_index_a**) וזאת באמצעות הפונקציה **`delay_train`**, החל מהתחנה שבה זוהתה ההתנגשות ובהתאם לכיוון נסיעתה (לפי הקלט **train_directions**) ולמשך העיכוב שנקבע לפי הקלט **delay**.
- לאחר כל עיכוב יש לבדוק מחדש את ההתנגשויות בין הרכבות, ולחזור על התהליך עד שלא נותרות התנגשויות.

שימו לב: טיפול בהתנגשות אחת עשוי ליצור התנגשויות חדשות בתחנות אחרות. לכן, יש לבצע בדיקה מחודשת של כלל ההתנגשויות לאחר כל עדכון בלוחות הזמנים.

לדוגמה:

1. קטע קוד:

```
>>> schedules = [[(0, 5), (10, 15)], [(3, 8), (12, 17)]]
>>> train_directions = ['ltr', 'ltr']
>>> delay = 5
>>> total_fixed = resolve_conflicts(schedules,train_directions,delay)
>>> print(schedules)
[[(10, 15), (20, 25)], [(3, 8), (12, 17)]]
>>> print(total_fixed)
2
```

רכבת	באר שבע מרכז	תל אביב סבידור מרכז
ltr: רכבת 0	(0, 5)	(10, 15)
ltr: רכבת 1	(3, 8)	(12, 17)

↓ resolve_conflicts ↓

רכבת	באר שבע מרכז	תל אביב סבידור מרכז
ltr: רכבת 0	**(10, 15)**	**(20, 25)**
ltr: רכבת 1	(3, 8)	(12, 17)

2. קטע קוד:

```
>>> schedules = [[(0, 5), (10, 15), (20, 25)], [(22, 27), (12, 17), (2, 7)], [(30, 35), (16, 18), (0, 3)]]
>>> train_directions = ['ltr', 'rtl', 'rtl']
>>> delay = 5
>>> total_fixed = resolve_conflicts(schedules,train_directions,delay)
>>> print(schedules)
[[(0, 5), (35, 40), (45, 50)], [(47, 52), (27, 32), (7, 12)], [(30, 35), (16, 18), (0, 3)]]
>>> print(total_fixed)
10
```

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
ltr: רכבת 0	(0, 5)	(10, 15)	(20, 25)
rtl: רכבת 1	(22, 27)	(12, 17)	(2, 7)
rtl: רכבת 2	(30, 35)	(16, 18)	(0, 3)

↓ resolve_conflicts ↓

רכבת	באר שבע מרכז	תל אביב סבידור מרכז	חיפה מרכז השמונה
ltr: רכבת 0	(0, 5)	**(35, 40)**	**(45, 50)**
rtl: רכבת 1	**(47, 52)**	**(27, 32)**	**(7, 12)**
rtl: רכבת 2	(30, 35)	(16, 18)	(0, 3)

חלק 3: ניהול חברת רכבות באמצעות מילון

בחלק זה תאגדו את נתוני חברת הרכבות במילון ותממשו פונקציות לניהול נתוני החברה ולתכנון נסיעות עבור קבוצות נוסעים. לצורך כך, תיעזרו בפונקציות שמימשתם בחלקים הקודמים. הימנעו משכפול קוד במימושכם (כלומר אין להעתיק את הלוגיקה של פונקציה קודמת לתוך פונקציה חדשה).

שימו לב- מומלץ לחכות לשיעור שיערך בתאריך **1.11** לפני שאתם ניגשים לפתרון החלק השלישי בתרגיל הבית.

מבנה מילון החברה

נתוני חברת הרכבות נשמרים במילון. המילון מכיל ארבעה מפתחות מטיפוס מחרוזת. כל מפתח מייצג סוג אחר של מידע על החברה וממופה לערך המתאים, כמפורט להלן:

- המפתח בעל הערך '**station_names**' ממופה לרשימת שמות התחנות לפי סדר העמודות במטריצה.
- המפתח בעל הערך '**schedules**' – ממופה לרשימת לוח הזמנים של הרכבות בחברה.
- המפתח בעל הערך '**train_directions**' – ממופה לרשימה של מחרוזות, שבה כל איבר מכיל את הערך $'ltr'$ או $'rtl'$ לפי כיוון רכבת.
- המפתח בעל הערך '**base_ticket_prices**' – ממופה לרשימת מחירי כרטיסים; מחיר לכל רכבת.

שימו לב: ניתן להניח כי הרשימות **schedules**, **train_directions** ו־**base_ticket_prices** שוות באורכן ומתאימות זו לזו לפי אינדקס הרכבת. כמו כן, אורך הרשימה **station_names** שווה למספר התחנות בכל אחד מלוחות הזמנים.

בנוסף, ניתן להניח כי:

- שמות תחנת המוצא ותחנת היעד המתקבלים בקלט לפונקציות קיימים ברשימה $company['station_names']$.
- חלון הזמן **departure_window** המתקבל בקלט לפונקציות תקין, כלומר הוא מכיל שני מספרים שלמים אי־שליליים ומתקיים $earliest_departure <= latest_departure$.

הדוגמה הבאה מציגה מילון המייצג את חברת הרכבות, והיא תשמש בכל סעיפי חלק 3.

```
>>> company = {
...     'station_names': [
...         'Beer Sheva Center',
...         'Tel Aviv Savidor Center,
...         'Haifa Merkaz HaShmona'
...     ],
...     'schedules': [
...         [(0, 5), (20, 25), (35, 40)],
...         [(47, 52), (27, 32), (7, 12)],
...         [(10, 15), (-1, -1), (45, 50)]
...     ],
...     'train_directions': ['ltr', 'rtl', 'ltr'],
...     'base_ticket_prices': [60.0, 55.0, 40.0]
... }
```

סעיף א': מציאת רכבות ישירות

בחלק 1 הוגדרה **נסיעה ישירה** כנסיעה מתחנת המוצא לתחנת היעד באמצעות רכבת אחת, ללא החלפה. בסעיף זה עליכם למצוא את כל הרכבות המאפשרות נסיעה ישירה בין שתי תחנות, ושזמן יציאתן מתחנת המוצא נמצא בחלון הזמן המבוקש, כולל גבולות החלון.

ממשו את הפונקציה:

$$find_direct_trains(company, start_station, destination_station, departure_window)$$

הפונקציה מקבלת בקלט:

- **company** - מילון המייצג חברת רכבות כפי שהוסבר בתחילת חלק 3.
- **start_station** - מחרוזת המייצגת שם תחנת המוצא שממנה הנוסע מעוניין לעלות לרכבת.
- **destination_station** - מחרוזת המייצגת שם של תחנת היעד אליה מעוניין להגיע הנוסע.
- **departure_window** – tuple המתאר את טווח הזמנים שבו הנוסע מעוניין למצוא זמן יציאה לרכבת. ה-tuple מורכב משני איברים (**earliest_departure, latest_departure**) כאשר:
  - **earliest_departure** – מספר אי-שלילי שלם המייצג את הזמן המוקדם (כולל) ביותר בו רכבת יכולה לצאת ועדיין להתאים לנוסע.
  - **latest_departure** – מספר אי-שלילי שלם המייצג את הזמן המאוחר ביותר (כולל) בו רכבת יכולה לצאת ועדיין להתאים לנוסע.

הפונקציה תחזיר רשימה של מילונים. כל מילון מתאר **אפשרות נסיעה ישירה** מתחנת המוצא לתחנת היעד העומדת בדרישות הנוסע. כל המפתחות במילון יהיו מטיפוס str. המילון יכיל בדיוק את המפתחות הבאים:

'**train_index**' – ממופה לערך מטיפוס int, המייצג את אינדקס הרכבת ברשימה $company['schedules']$.
'**departure_time**' – ממופה לערך מטיפוס int, המייצג את זמן היציאה של הרכבת מתחנת המוצא.
'**arrival_time**' – ממופה לערך מטיפוס int, המייצג את זמן ההגעה של הרכבת לתחנת היעד.
'**travel_time**' – ממופה לערך מטיפוס int, המייצג את משך הנסיעה מתחנת המוצא לתחנת היעד.
'**base_price**' – ממופה לערך חיובי מטיפוס float, המייצג את מחיר הבסיס של כרטיס נסיעה ברכבת.

אם לא נמצאה אפשרות מתאימה לנסיעה ישירה, הפונקציה תחזיר רשימה ריקה.

שימו לב: סדר אפשרויות הנסיעה הישירה ברשימה המוחזרת אינו חשוב.

לדוגמה:
קטע קוד:

```
>>> # company is defined above
>>> start_station = 'Beer Sheva Center'
>>> destination_station = 'Haifa Merkaz HaShmona'
>>> departure_window = (0, 20)
>>> print(find_direct_trains(company,start_station,destination_station,departure_window))
[
        {
                'train_index':0,
                'departure_time':5,
                'arrival_time':35,
                'travel_time':30,
                'base_price':60.0
        },
        {
                'train_index':2,
                'departure_time':15,
                'arrival_time':45,
                'travel_time':30,
                'base_price':40.0
        }
]
```